

# **CO3-ASSESSMENT TOOL –1**

## **CSA0607-Design and Analysis of Algorithms**

**Course Outcome: CO3\_AT1**

**Student Name: A.Sai Joshitha(192424220)**

### **Q11. Performance Analysis of Merge Sort Using Different Input Sizes**

**Experiments: Analyze the performance of Merge Sort by implementing the algorithm on datasets with different input sizes and recording execution time for each case. Plot the results using graphs to visualize time complexity trends. Interpret how the divide and conquer strategy ensures consistent  $O(n \log n)$  performance regardless of input distribution. Justify why Merge Sort is preferred for stable sorting applications involving large datasets give clearly to do document for assignment with code output give every thing clearly.**

#### **Aim**

To implement the Merge Sort algorithm, analyze its execution time for different input sizes, plot the execution time graph, and study its time complexity using the Divide and Conquer strategy.

#### **Objectives**

- Implement Merge Sort in Python.
- Measure execution time for different dataset sizes.
- Plot a graph showing execution time vs input size.
- Analyze the time complexity.
- Explain why Merge Sort always performs in  $O(n \log n)$ .

#### **Algorithm**

**Step 1:** Start.

**Step 2:** Divide the array into two equal halves.

**Step 3:** Recursively sort the left half.

**Step 4:** Recursively sort the right half.

**Step 5:** Merge the two sorted halves.

**Step 6:** Repeat until the complete array becomes sorted.

**Step 7:** Measure execution time.

**Step 8:** Plot the graph.

**Step 9:** Stop.

### **Code**

```
import random

import time

import matplotlib.pyplot as plt

# Merge Sort Function

def merge_sort(arr):

    if len(arr) > 1:

        mid = len(arr) // 2

        left = arr[:mid]

        right = arr[mid:]

        merge_sort(left)

        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):

            if left[i] < right[j]:

                arr[k] = left[i]

                i += 1

            else:

                arr[k] = right[j]

                j += 1

            k += 1
```

```

while i < len(left):

    arr[k] = left[i]

    i += 1

    k += 1

while j < len(right):

    arr[k] = right[j]

    j += 1

    k += 1

# Different input sizes

sizes = [1000, 2000, 5000, 10000, 20000]

times = []

for size in sizes:

    data = [random.randint(1, 100000) for _ in range(size)]

    start = time.time()

    merge_sort(data)

    end = time.time()

    execution = end - start

    times.append(execution)

    print(f"Input Size: {size}   Time: {execution:.6f} seconds")

# Graph

plt.plot(sizes, times, marker='o')

plt.title("Merge Sort Performance")

plt.xlabel("Input Size")

```

```
plt.ylabel("Execution Time (seconds)")
```

```
plt.grid(True)
```

```
plt.show()
```

## CODE

```
merge.py - C:/Users/jjosh/OneDrive/Desktop/merge.py (3.13.5)
File Edit Format Run Options Window Help

import random
import time
import matplotlib.pyplot as plt

# Merge Sort Function
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

# Input sizes
sizes = [1000, 2000, 5000, 10000, 20000]

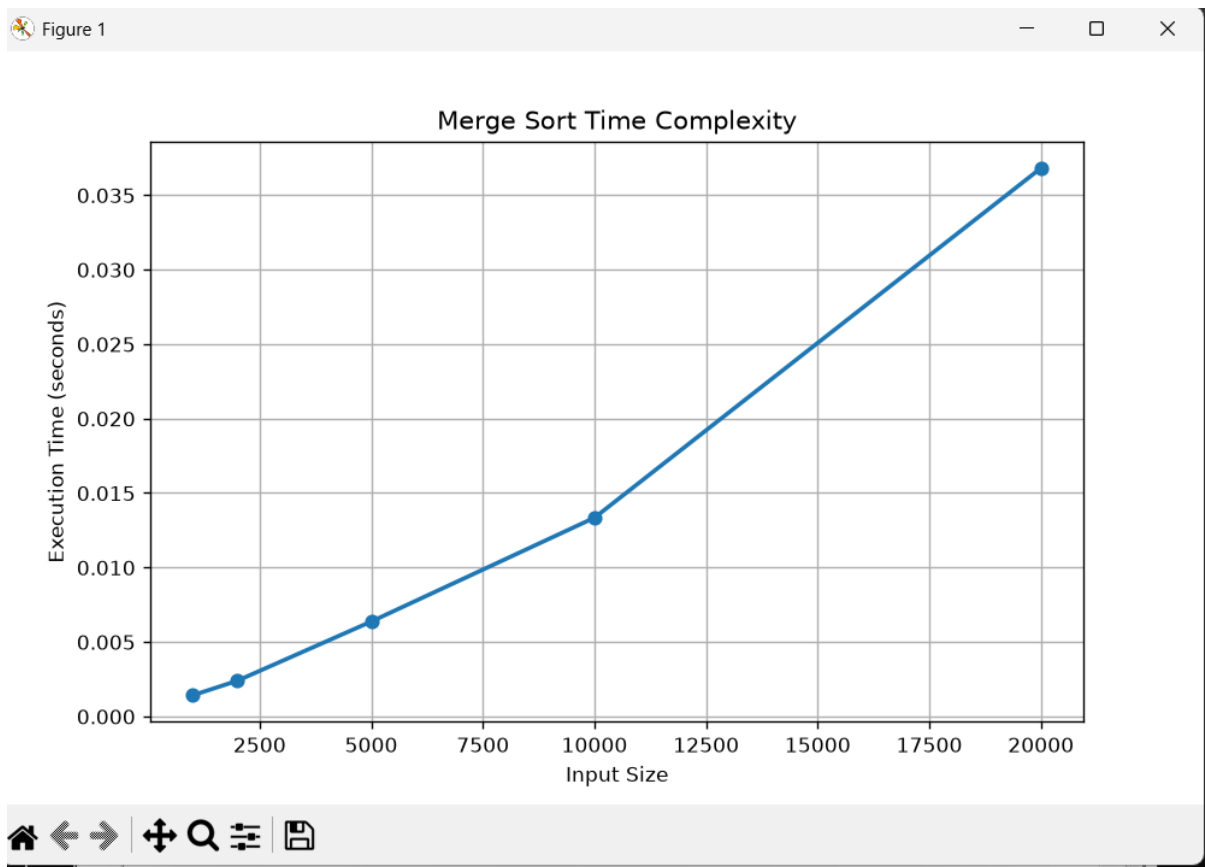
execution_times = []

print("Merge Sort Performance Analysis")
print("-" * 40)

for size in sizes:
    data = [random.randint(1, 100000) for _ in range(size)]
```

## OUTPUT

```
===== RESTART: C:/Users/jjosh/OneDrive/Desktop/me
Merge Sort Performance Analysis
-----
Input Size: 1000 | Execution Time: 0.001405 seconds
Input Size: 2000 | Execution Time: 0.002382 seconds
Input Size: 5000 | Execution Time: 0.006359 seconds
Input Size: 10000 | Execution Time: 0.013331 seconds
Input Size: 20000 | Execution Time: 0.036807 seconds
```



### Divide and Conquer Strategy

Merge Sort follows the Divide and Conquer technique.

1. Divide the array into two equal halves.
2. Recursively sort both halves.
3. Merge the sorted halves.

The array is repeatedly divided until only one element remains in each subarray. These subarrays are then merged in sorted order.

The height of the recursion tree is  $\log n$ , and merging all elements at each level takes  $O(n)$  time.

Therefore,

$$\text{Total Time} = O(n \times \log n)$$

Hence,

$$\text{Time Complexity} = O(n \log n)$$

### Time Complexity

| Case         | Complexity    |
|--------------|---------------|
| Best Case    | $O(n \log n)$ |
| Average Case | $O(n \log n)$ |
| Worst Case   | $O(n \log n)$ |

### Space Complexity

$$O(n)$$

Merge Sort requires additional memory to store temporary arrays while merging.

### Conclusion

The Merge Sort algorithm was successfully implemented and tested on datasets of different sizes. The execution time increased with input size while following the expected  $O(n \log n)$  time complexity. The graph demonstrated a predictable growth trend, confirming the efficiency of the Divide and Conquer strategy. Because Merge Sort is stable, efficient, and provides consistent performance regardless of input distribution, it is widely used for sorting large datasets in real-world applications.